

CMPSC 132 – Final Project Guidelines

Project Overview

The goal of the final project is to apply Python programming and data structures concepts. You will build a complete working program, connect your GitHub to Gradescope, a 1-2, page report, demonstrate your project in person (Sign up sheet will be provided).

Enhancing any of the projects listed below may provide an opportunity for extra credit. Please email me with your idea for approval before beginning the work on the project.

You must choose from **one** of the following projects and email me your choice by **Friday, 4/10/26, 5 pm.**

Projects

1. Tic-Tac-Toe Game (2-Player, Terminal-Based)

Description

Build a two-player Tic-Tac-Toe game that runs in the terminal. Two players take turns entering their moves until one of them wins or the game ends in a draw.

Requirements

- Display a **3×3 board**
- Players take turns (**X and O**)
- Accept user input for row and column (0–2 or 0–2)
- Prevent invalid moves (e.g., selecting an already occupied cell) or co-ordinates outside the board
- Detect:
 - Winning conditions (rows, columns, diagonals)
 - Draw condition (board full, no winner)

Implementation Details

- Use a **list of lists** to represent the board
- Some of the functions:
 - `print_board(board)`
 - `check_winner(board, player)`
 - `is_draw(board)`
- Alternate turns until game ends

2. Snake & Ladder Game (2-Player, Terminal-Based)

Description

Create a Snake & Ladder game where two players roll a dice and move along a board from 1 to 100. Players climb ladders and slide down snakes.

Requirements

- Simulate dice roll using: `random.randint(1, 6)` to determine how many spaces to advance.
- Players move from **1 to 100**
- **Snake Interactions:** If a player lands on a cell containing a snake's head, they must slide down to the cell where its tail is located.
- **Ladder Interactions:** If a player lands on a cell with the bottom of a ladder, they automatically climb up to the top of the ladder cell.
- **Cells:** Players can occupy the same cell simultaneously without affecting each other.
- Alternate turns
- **Winning:** Player must reach **exactly 100** to win

Implementation Details

- Use dictionaries to store snake and ladder mappings

```
snakes = {16: 6, 48: 30}
ladders = {3: 22, 20: 38}
```

- Track positions of both players
- Define rule for overshooting 100 (recommended: player stays in same position)

3. Number Guessing Game (Single Player, Terminal-Based)

Description

A single player, terminal based number guessing game where the computer generates a random number and the player attempts to guess based on feedback. After each guess, the program gives feedback such as “Too High” or “Too Low” until the correct number is guessed.

This is a *simple* but effective project to practice logic building, loops, and input handling.

Requirements

- **Random Number Generation:** Game starts by a random number generation by the program (e.g., between 1 and 100)
- **User Input:** Player guesses the number (using the `input()` function and convert to `int()`).
User is allowed multiple attempts to guess the number
- Provide feedback:
 - “Too high”: If the guess is greater than the number generated, it prompts the user to guess a lower number.
 - “Too low”: If the guess is lower than the number generated, it prompts the user to guess a higher number.
 - “Correct!”: If the guess matches, game displays “Congratulations!” and ends the game.
- **Attempt Tracking:** Program should keep track of the number of attempts
- End the game when the correct number is guessed

Implementation Details

- Use: `random.randint(1, 100)`
- Use a loop to continue until the correct guess
- Validate user input (ensure it is a number)
- Store and display the number of attempts

Suggested Enhancements (Optional)

- Difficulty levels (easy, medium, hard)
- Limit number of attempts
- Allow replay

4. Web Scraping Project (Laptop Price Tracker)

Description

Web scraping projects for Create a program that extracts laptop details (name, price, etc.) from a website. This project introduces you to web scraping and working with real-world data.

*******This project may be challenging*******

Requirements

- Extract the following:
 - Laptop name
 - Price
 - Ratings (if available)
- Display the data in a clean format in the terminal

Implementation Details

- Use libraries:

```
import requests from bs4 import BeautifulSoup
```

- Steps:
 - Send a request to a website using `requests.get()`
 - Parse the HTML using `BeautifulSoup`
 - Identify HTML tags containing:
 - Product name
 - Price
 - Extract and print the data

Important Notes (Simplified for Students)

- Choose simple websites (static pages)
- Avoid websites that block scraping
- Do NOT scrape sensitive or restricted content

Example Output

Laptop: Dell Inspiron 15

Price: \$650

Laptop: HP Pavilion

Price: \$720

General Requirements (All Projects)

Code Requirements

- Use **functions** *and/or* **classes** to organize your code
- Use appropriate **data structures** (lists, dictionaries, etc.)
- Validate all user inputs
- Ensure your code runs without errors
- Handle errors and edge cases

1. GitHub (Required)

You must submit a **GitHub repository** containing:

- All source code files
- A README .md file including:
 - Project description
 - Instructions to run the program
- Proper commits (not just one upload)

2. Submission Instructions

- Upload your code in GitHub, link your [GitHub to Gradescope](#) and select the 'master' branch and click on Upload.
- Please note that any changes made to the 'master' branch in GitHub, **DOESN'T automatically** reflect in Gradescope. After the commit in GitHub, please redo the above step in Gradescope to update your code.
- The deadline is ***Tuesday, April 28th 11:59 pm in Gradescope under Final Project.***
- Submit a 2- 3 page report (PDF or Word) in **Gradescope** under **Final Project**.
- *You have to demonstrate your completed work on either April 29th or 30th. A sign up sheet will be provided. Demonstration of your work can be between 3-5 minutes. This will be in the TA office, 300 West College Avenue.*

a. Final Report (1–2 Pages)

Your report must include:

- Project description
- Approach and logic used
- Data structures used
- Challenges faced
- Possible improvements

Grading Rubric (100 Points Total)

Criteria	Points	Description
Functionality	40	Program works correctly and meets all requirements
Code Organization	20	Well-structured code with functions
Data Structures	15	Effective use of lists/dictionaries
Code Readability & Comments	10	Clean, readable, well-commented code
Edge Case Handling	10	Handles invalid input and errors
Creativity / Extra Features	5	Additional features beyond requirements
Enhancements for Extra credit	Depends on the work	Please email me prior to working on the enhancements of the project